Introduction to Linux Systems

GNU Make

Chia-Heng Tu
Dept. of Computer Science and Information Engineering
National Cheng Kung University
Fall 2025

Lab

- Build a simple C project structure.
- Learn how to separate **<u>header</u>** and the **<u>source</u>** file.
- Use Makefile for automated compilation.
- Practice using `.gitignore` to avoid tracking unnecessary files.

https://hiltmon.com/blog/2013/07/03/a-simple-c-plus-plus-project-structure/

New folder for this lab

- Install required packages
  ```
  $ sudo apt install gcc tree make
  ```
- Create a working directory

  ```
  $ cd <Your local git repo>
  $ mkdir lab_5_<student_ID>
  $ cd lab_5_<student_ID>
  ```

Add .gitignore to prevent tracking object files

```
# Ignore the build and lib dirs
build
lib/*

# Ignore any executables
bin/*

# Ignore temporary files
*.swp
```

```
├── .gitignore
├── include/
│   └── reverse.h
├── Makefile
└── src/
    ├── reverse.c
    └── main.c
```

Split function into different file (Header)

```
#pragma once

char *reverse(char *dest, const char *src);
```

or

```
#ifndef REVERSE_H
#define REVERSE_H

char *reverse(char *dest, const char *src);

#endif /* REVERSE_H */
```

```
├── .gitignore
├── include/
│   ├── reverse.h
├── Makefile
└── src/
    ├── reverse.c
    └── main.c
```

Note: include guard

https://en.wikipedia.org/wiki/Pragma_once
https://en.wikipedia.org/wiki/Include_guard

Split function into different file (Source)

```
#include "../include/reverse.h"
char *reverse(char *dest, const char *src)
{
        //FIX ME
}
```

```
├── .gitignore
├── include/
│   └── reverse.h
├── Makefile
└── src/
    ├── reverse.c
    └── main.c
```

Program entry point

```
#include <stdio.h>
#include <stdlib.h>
#include "../include/reverse.h"
int main()
{
    const char *src = "F76123456";
    char *dst = malloc(10);
    dst = reverse(dst, src);
    printf("%s", dst);
    return 0;
}
```

Change it to your student ID number!

```
├── .gitignore
├── include/
│   └── reverse.h
├── Makefile
└── src/
    ├── reverse.c
    └── main.c
```

Makefile

```
CC := gcc
SRCDIR := src
BUILDDIR := build
BINDIR := bin
INCDIR := include
TARGET := $(BINDIR)/runner

SRCEXT := c
SOURCES := $(shell find $(SRCDIR) -type f -name *.$(SRCEXT))
HEADERS := $(shell find $(INCDIR) -type f -name *.h)
OBJECTS := $(patsubst $(SRCDIR)/%,$(BUILDDIR)/%,$(SOURCES:.$(SRCEXT)=.o))
CFLAGS := -O2 -Wall

all: $(TARGET)

$(TARGET): $(OBJECTS)
             mkdir -p $(BINDIR)
             @echo "> Linking..."
             $(CC) $^ -o $(TARGET)


$(BUILDDIR)/%.o: $(SRCDIR)/%.$(SRCEXT)
             mkdir -p $(BUILDDIR)
             @echo "> Compiling..."
             $(CC) $(CFLAGS) -c -o $@ $<

clean:
             @echo "> Cleaning...";
             $(RM) -rf $(BUILDDIR) $(TARGET)
```

```
├── .gitignore
├── include/
│   └── reverse.h
├── Makefile
└── src/
    ├── reverse.c
    └── main.c
```

Variable and alias in Makefile

- `SOURCES := $(shell find $(SRCDIR) -type f -name *.$(SRCEXT))`
  - `src/reverse.c src/main.c`
- `HEADERS := $(shell find $(INCDIR) -type f -name *.h)`
  - `include/reverse.h`
- `OBJECTS := $(patsubst $(SRCDIR)/%,$(BUILDDIR)/%,$(SOURCES:.$(SRCEXT)=.o))`
  - `$(SOURCES:.$(SRCEXT)=.o) = src/reverse.o src/main.o`
  - `build/reverse.o build/main.o`
- `$(CC) $(CFLAGS) -c -o $@ $<`
  - `gcc -O2 -Wall -c -o build/reverse.o src/reverse.c`
  - `gcc -O2 -Wall -c -o build/main.o src/main.c`
- `main.o: main.c a.o`
  `$(CC) $(CFLAGS) -c -o $@ $<`

```
├── .gitignore
├── include/
│   └── reverse.h
├── Makefile
└── src/
    ├── reverse.c
    └── main.c
```

Current file architecture

```
$ tree .
.
├── include/
│   └── reverse.h
├── Makefile
└── src/
    ├── reverse.c
    └── main.c
```

Use Makefile and execute program

```
$ make
mkdir -p build
> Compiling...
gcc -O2 -Wall -c -o build/main.o src/main.c
mkdir -p build
> Compiling...
gcc -O2 -Wall -c -o build/encrypt.o src/encrypt.c
mkdir -p bin
> Linking...
gcc build/main.o build/encrypt.o -o bin/runner

$ ./bin/runner
 65432167F
```

File architecture after using make

```
$ tree .
.
├── bin/
│   └── runner
├── build/
│   ├── reverse.o
│   └── main.o
├── include/
│   └── reverse.h
├── Makefile
└── src/
    ├── reverse.c
    └── main.c
```

Demo

- Complete the code and run the `make` command to compile the project.
- Execute the compiled executable file.
- Use the `tree` command to display the file structure.

繳交作業需包含以下部分：
1. 題目要求截圖（如右圖）
2. 本次作業/課程心得

將上述內容彙整成一份 PDF 檔案

格式：**HW5_*StudentID*.pdf**
e.g., **HW5_F74131234.pdf**

```
joe@Evan-Lab:~/linux_lab$ make
mkdir -p build
> Compiling...
gcc -O2 -Wall -c -o build/main.o src/main.c
mkdir -p build
> Compiling...
gcc -O2 -Wall -c -o build/reverse.o src/reverse.c
mkdir -p bin
> Linking...
gcc build/main.o build/reverse.o -o bin/runner
joe@Evan-Lab:~/linux_lab$ ./bin/runner
65432167F
joe@Evan-Lab:~/linux_lab$
```

```
joe@Evan-Lab:~/linux_lab$ tree .
.
├── bin
│   └── runner
├── build
│   ├── main.o
│   └── reverse.o
├── include
│   └── reverse.h
├── Makefile
└── src
    ├── main.c
    └── reverse.c

4 directories, 7 files
```

QUESTIONS